# Perceptual Local Collision-Free Motion Generation for Manipulators

Guangbao Zhao, Jianhua Wu, *Member, IEEE*, and Zhenhua Xiong, *Member, IEEE*

***Abstract*—Collision-free motion planning has a well-established research history, yet the majority of studies have been centered around Cartesian space and assume the exact positions and shapes of the obstacles in the environment are known. In this letter, we introduce a perceptual local reactive motion controller for manipulators that navigates to desired configurations while avoiding collisions with both static and dynamic obstacles, as well as self-collisions. We formulate collision-free motion generation as a constrained quadratic programming problem, where collision avoidance is achieved by linearizing minimum distance constraints. We have integrated obstacle velocity into the planning algorithm to ensure reliable collision avoidance in the presence of dynamic obstacles. To enable fast collision distance queries in high-dimensional configuration space, we approximate implicit signed distance functions using neural networks and leverage forward kinematics to eliminate non-linear mappings between configuration and Cartesian spaces. This approach improves both computational efficiency and fitting accuracy. The gradients of the distance function with respect to joint states are provided for collision constraints using the derived analytical gradient. Additionally, self-collision avoidance is achieved using a similar method. The proposed planner typically solves within 10 milliseconds, even in scenarios with 10,000 points. Both simulation and physical experiments validate the effectiveness of our approach.**

***Index Terms*—Analytical gradient, constrained quadratic programming, distance regression, reactive collision-free motion planning.**

## I. Introduction

REACTIVE collision-free motion planning is essential for manipulators to execute tasks in dynamic or partially known environments. Despite numerous advances in motion planning, generating fast and reliable collision-free motion for high degree of freedom robots in dynamic environments remains an unresolved challenge.

Received 19 May 2024; accepted 11 November 2024. Date of publication 21 November 2024; date of current version 2 December 2024. This article was recommended for publication by Associate Editor Rahul Shome and Editor Hanna Kurniawati upon evaluation of the reviewers' comments. This work was supported in part by Jiangsu Provincial Science and Technology Department Program Special Funds (Basic Research on Frontier Leading Technologies) under Grant SBK2023050162, in part by the Science and Technical Innovation 2030-Artifcial Intelligence of New Generation under Grant 2018AAA0102704, and in part by the State Key Laboratory of Mechanical System and Vibration under Grant MSVZD202205. *(Corresponding author: Jianhua Wu.)*

The authors are with the State Key Laboratory of Mechanical System, and Vibration, Institute of Robotics, School of Mechanical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: zhaoguangbao@sjtu.edu.cn; wujh@sjtu.edu.cn; mexiong@sjtu.edu.cn).

This letter has supplementary downloadable material available at https://doi.org/10.1109/LRA.2024.3504316, provided by the authors.

Digital Object Identifier 10.1109/LRA.2024.3504316

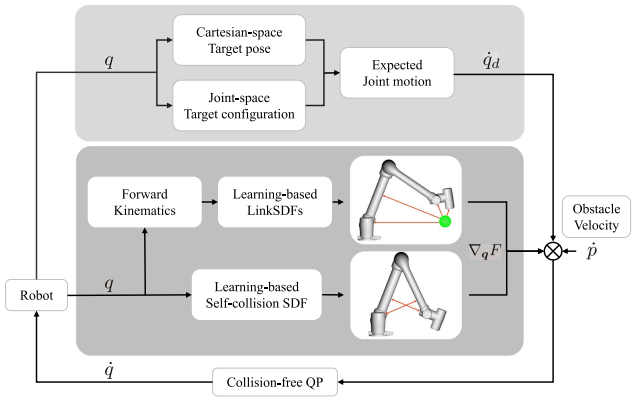


Fig. 1. Architecture of our proposed collision-free motion planner. We reformulate it as a QP problem, where collision avoidance constraints are constructed using neural networks that represent the robot's signed distance functions for both environmental and self-collisions.

The purpose of this letter is to reactively generate collision-free motion from point to point in both configuration space and Cartesian space, while simultaneously avoiding collisions between the robot's links and the environment, as well as preventing self-collisions. Some sampling-based methods [1], [2] have demonstrated state-of-the-art performance in dynamic environments. Nevertheless, these approaches still encounter challenges in achieving real-time reactive motion generation, primarily due to the computational complexity involved in sampling rollouts and extensive collision checking. This problem is exacerbated when estimating the distance to obstacles and their gradient.

To address these challenges, we propose a reactive scheme for high degree of freedom manipulators. We formulate collision-free motion planning as a constrained Quadratic Programming (QP) problem in the configuration space. Our approach integrates both joint velocity and obstacle velocity to handle environmental collisions. The inclusion of obstacle velocity is proven to be essential to ensure that no collisions occur. Additionally, we introduce a dynamic adjustment strategy for distance hyperparameters based on proximity to local minima, which improves the performance of collision-free planning. The architecture of our approach is illustrated in Fig. 1.

To construct collision-avoidance constraints directly from perceptual data, we employ neural networks to learn implicit distance functions for the robot, which are referred to as link SDFs (Signed Distance Functions). While previous work [3] has utilized neural networks to represent implicit distance functions,

the non-linear mapping from configuration space to Cartesian space makes it challenging to achieve high regression accuracy. The work [4] improved regression accuracy through the use of forward kinematics (FK). However, it did not explicitly provide the gradient of the distance function with respect to the joint states, thereby limiting its applicability in gradient-based optimization methods. To overcome these limitations, we derive the analytical gradient of the distance function, which guides us in regressing both the distance and the unit vector between witness point pairs. This approach not only enables the incorporation of forward kinematics to enhance regression accuracy but also provides the necessary analytical gradients for constructing environmental collision constraints. Furthermore, we extend the method to include self-collision avoidance while maintaining high reactivity.

We demonstrate the effectiveness of our approach through both simulation and physical experiments, in static environments with multiple objects and scenarios with moving obstacles. The reactive collision-free motion generation can operate at 100 Hz even in environments containing 10,000 points. In summary, the contributions of this letter are as follows:

1) We propose a real-time collision-free motion generation method that considers both joint and obstacle velocities in the QP formulation. It can generate collision-free motion in both Cartesian and joint spaces for robots in dynamic scenarios based on perceptual data.
2) We propose a forward-kinematics-based method to simultaneously provide precise regression of implicit distance functions and their gradients with respect to joint states, which are used to construct environmental collision constraints. Furthermore, we incorporate these constraints and self-collision constraints into the QP framework, preventing both environmental and self-collisions.

## II. Related Works

### A. Reactive Motion Planning

Classical motion planning methods such as STOMP [5], CHOMP [6], and Trajopt [7] are typically limited to open-loop planning. An approach to reactive motion control is potential fields, where the robot is repelled from obstacles [8]. However, designing an appropriate potential function is challenging. Haviland et al. [9] reformulated reactive motion planning as a quadratic programming problem and used the differential inverse kinematics to choose instantaneous joint velocities to maximize manipulability and avoid environmental collisions. However, it can only achieve point-to-point motion in Cartesian space. In addition, it can only compute distances based on mesh models and cannot be directly applied in environments represented by point clouds. To prevent local minima encountered in gradient-based approaches, sampling-based stochastic optimization methods [2], [10], [11], [12] have been proposed. Nevertheless, due to the massive sampled rollouts, these approaches still face challenges in achieving real-time reactive motion generation. Sundaralingam et al. [13] utilized massively parallel GPU computation to accelerate collision-free motion generation.

### B. Environment Collision Avoidance

Collision checking is fundamental for motion planning. In [9], Pybullet was adopted to compute the distance between triangulated meshes. However, this approach becomes computationally demanding in complex environments. Signed Distance Functions (SDFs) have garnered considerable attention in collision checking due to their notable query efficiency and capability to accurately describe complex shapes [14], [15]. In [5] and [6], pre-computed signed distance fields and numerical gradients are utilized in static environments. A novel spatial data structure was presented in [16] to accelerate collision queries between robots and point clouds. With the emergence of learning-based methods, researchers have begun exploring the potential of learning collision models from data. To efficiently handle dynamic scenes, SDFs are used to represent robots instead of scenes. Liu et al. [17] proposed regularized deep SDFs with neural networks, integrating them into whole-body motion control based on artificial potential fields to achieve collision avoidance. Vasilopoulos et al. [18] utilized forward kinematics to generate control points on the robot's surface and computed their signed distances to point clouds using GPU acceleration. However, due to the computational demand, this method is only suitable for handling a smaller number of control points and scene points. The neural joint space SDFs proposed in [3] enable querying distance values with points and joint configurations as input and utilize automatic differentiation to obtain their gradients. The kinematic chain was exploited in [4] to simplify the fitting problem for robot links. However, it does not provide the gradient with respect to joint configurations. To guarantee safe motion planning under sensing uncertainty, an implicit neural model that accounts for sensor noise was proposed in [19].

### C. Self-Collision Avoidance

Due to the presence of multiple links in serial robots, self-collision avoidance is crucial during motion planning. However, handling multiple links results in expensive self-collision computations. Support vector machines have been widely applied to learn a continuously differentiable self-collision boundary function in joint space [20], [21], [22]. This boundary is then used to formulate the collision constraints within a quadratic programming framework. Noël et al. [23] addressed self-collision in legged robots by using simplified geometries to handle pairs of links that are far from each other and applied a multi-layer perceptron to approximate the joint-space distance field for links that are close to each other. In [2], positional encoding of joint configuration was used as input to neural networks to predict the closest distance between links. Subsequently, the cost of self-collision was incorporated into a sampling-based optimization algorithm.

## III. Gradient-Based Collision-Free QP

### A. Preliminaries

The damped least squares algorithm [24] is a classical strategy for differential inverse kinematics problems, which selects instantaneous joint velocities to drive the robot from the current

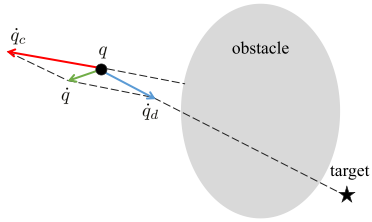


Fig. 2. Decomposition of the actual velocity. The actual velocity $\dot{\boldsymbol{q}}$ is composed of: the default velocity $\dot{\boldsymbol{q}}_d$, which is attracted by the target, and the constrained velocity $\dot{\boldsymbol{q}}_c$, which is repelled by the obstacle.

pose to the target pose while minimizing the norm of the joint velocities. It can be reformulated as an unconstrained quadratic programming problem:

$$\min \quad \|J\dot{q} - \boldsymbol{e}^*\|^2 + \lambda^2\|\dot{q}\|^2 \tag{1}$$

where $\boldsymbol{e}^* = \log({}^wT_e\,{}^wT_c{}^{-1}) \in se(3)$, ${}^wT_e, {}^wT_c \in SE(3)$ denote the expected and current poses respectively, and $\lambda \in R$ is a non-zero damping constant.

The classical potential field method integrates virtual repulsive forces from obstacles and attractive forces to guide the robot toward the goal. Inspired by this, we decompose the actual velocity $\dot{\boldsymbol{q}}$ into two components: the desired velocity $\dot{\boldsymbol{q}}_d$ and the constrained velocity $\dot{\boldsymbol{q}}_c$. For a planar point, its configuration space is equivalent to Euclidean space. The robot is repelled by obstacles with $\dot{\boldsymbol{q}}_c$ and attracted to the target with $\dot{\boldsymbol{q}}_d$, as shown in Fig. 2. Then, the formulation (1) can be reformulated as:

$$\min_{\dot{\boldsymbol{q}}_c} \|\dot{\boldsymbol{q}}_d + \dot{\boldsymbol{q}}_c\|_Q + \|J\dot{\boldsymbol{q}}_c\|_R \tag{2}$$

where $J\dot{\boldsymbol{q}}_c$ denotes the deviation in Cartesian space introduced by $\dot{\boldsymbol{q}}_c$. The formulation includes a slack variable $\dot{\boldsymbol{q}}_c$ compared to the form (1), effectively introducing additional redundancy to the problem, which is especially useful for robots with six or fewer degrees of freedom. Additionally, it provides a way to introduce reference velocity input $\dot{\boldsymbol{q}}_d$, similar to the attractor in potential field methods. This approach searches for an appropriate constrained velocity through optimization instead of relying on a well-designed potential function. The optimization objective of problem (2) can also be formulated as $\|\dot{\boldsymbol{q}}_c\|_Q + \|J\dot{\boldsymbol{q}}_c\|_R$ in Cartesian space or $\|\dot{\boldsymbol{q}}_c\|_Q$ in configuration space.

### B. Modeling Environment Collision

We consider a serial-links robot with $n$ joints, $\boldsymbol{q} = (q_1, q_2, \ldots, q_n) \in \mathbb{R}^n$ and $m$ links. In this letter, the left superscript indicates the coordinate frame in which the vector is represented. For the sake of description, the left superscript is omitted when the vector is expressed in the base coordinate frame. Given a point $p \in \mathbb{R}^3$ where the nearest point on the $k$th link to the point is $p_k$, the minimal distance between the $k$th link and $p$ is given by $d_k(p_k, p) = ((p_k - p)^T(p_k - p))^{\frac{1}{2}}$. Therefore, its derivatives with respect to $\boldsymbol{q}$ and $p$ are:

$$\frac{\mathrm{d}d_k}{\mathrm{d}\boldsymbol{q}} = \frac{\partial d_k}{\partial p_k}\frac{\partial p_k}{\partial \boldsymbol{q}} = \frac{1}{d_k}(p_k - p)^T J_k(\boldsymbol{q}) = n_k^T J_k(\boldsymbol{q}) \tag{3}$$

$$\frac{\mathrm{d}d_k}{\mathrm{d}p} = \frac{1}{d_k}(p - p_k)^T \tag{4}$$

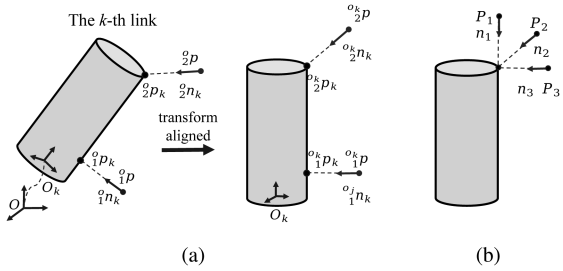


Fig. 3. (a) Forward kinematics is adopted to transform the query point and align it with the coordinate frame of each link. (b) The unit vector between the witness point pairs differs from the normal vector of the nearest point on the links.

where $J_k(\boldsymbol{q}) \in \mathbb{R}^{3\times n}$ represents the translation Jacobian matrix of the point $p_k$ with respect to $\boldsymbol{q}$, and $n_k = \frac{p_k - p}{d_k} \in \mathbb{R}^3$ is unit direction vector from $p$ to $p_k$.

The calculation of the nearest point $p_k$ is critical and time-consuming. We aim to learn a regression function $NN_{\text{links}}(\boldsymbol{q}, p)$ to approximate an $m$-entry vector, where each entry is $[\Gamma_k, n_k^T]^T \in \mathbb{R}^4$, where $\Gamma_k$ is the predicted minimal distance between the $k$th link and $p$, and $n_k$ is the predicted unit direction vector from $p$ to $p_k$. Therefore, $p_k = n_k\Gamma_k + p$. We define $\Gamma_{\text{env}}(\boldsymbol{q}, p) = [\Gamma_1(\boldsymbol{q}, p), \ldots, \Gamma_m(\boldsymbol{q}, p)]^T \in \mathbb{R}^m$ to represent the predicted minimum distance vector between the robot links and the query point $p$. Therefore,

$$\nabla_{\boldsymbol{q}}\Gamma_{\text{env}} = [\nabla_{\boldsymbol{q}}\Gamma_1, \nabla_{\boldsymbol{q}}\Gamma_2, \ldots, \nabla_{\boldsymbol{q}}\Gamma_m] \in \mathbb{R}^{n\times m}$$

where $\nabla_{\boldsymbol{q}}\Gamma_k = J_k^T(p_k)n_k \in \mathbb{R}^n, k = 1, \ldots, m$.

Instead of directly learning a holistic SDF network $NN_{\text{env}}(\boldsymbol{q}, p) : R^n \times R^3 \to R^{4m}$ from the configuration space, we first leverage forward kinematics to transform the query point $p$, aligning it with the coordinate frame of each link ${}^{o_j}p = \text{Fk}(\boldsymbol{q})p$, where $\text{Fk}(\cdot)$ calculates the transformation matrix between the coordinate frames of the links and the base, as illustrated in Fig. 3(a). For the $k$th link, we can then employ an MLP network to approximate its signed distance function $NN_{\text{link},k}({}^{o_k}p) : R^3 \to R^4$. The link SDFs $NN_{\text{links}}$ are composed of each link SDF, $NN_{\text{links}} = [NN_{\text{link},1}, NN_{\text{link},2}, \ldots, NN_{\text{link},m}]$. The advantage of using $NN_{\text{links}}$ over $NN_{\text{env}}$ is that $NN_{\text{links}}$ depends only on the link's geometry, whereas $NN_{\text{env}}$ depends on both the joint states and the link's geometry. Moreover, due to the non-linear mapping from configuration space to Cartesian space, regressing $NN_{\text{env}}$ with high precision is more challenging. This is validated in Section IV-B. By explicitly using forward kinematics, we avoid learning the non-linear mapping between the configuration space and Cartesian space.

A suitable dataset is critical for learning accurate SDFs. We synthesize the dataset using the 3D mesh model of links. Each sample data includes a query point $p$, the corresponding nearest point on the link $p_k$, the minimal distance $d$, and the unit vector $n$ between the witness point pairs $(p, p_k)$. It is worth noting that the unit vector is different from the normal of $p_k$, as shown in Fig. 3(b). We scale the bounding box of the link model, as illustrated in Fig. 4, and then uniformly sample within the scaled bounding box. We sampled 10,000, 10,000, 30,000, 30,000,

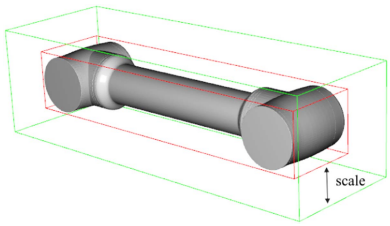


Fig. 4. Scaled bounding boxes for sampling points. The red box represents the bounding box of the link mesh. We adopt different distance scales to generate scaled bounding boxes (shown in green), which limit the range of the sampling query points.

and 50,000 data sets at scales of 0.05, 0.1, 0.2, 0.5, and 2.0 meters, respectively, to more effectively concentrate points near the surface of the link.

The network structure is determined by performing a grid search, whose detail is shown in Section IV-A. The final structure is composed of 5 fully connected layers, with all hidden layers being 64-dimensional. The ReLU function is chosen as the activation function. The loss function is defined as $\text{loss} = \omega\text{mse}(d, d^*) + \text{mean}(1 - n^T n^*)$. The first term penalizes distance errors, while the second penalizes gradient vector alignment errors, with $\omega$ as a constant balancing the two loss terms.

### C. Modeling Self-Collision

For each configuration state $\boldsymbol{q}$, there exists a unique minimal distance between the links of the robot. Hence, an implicit self-collision distance function exists. We adopt the same network architecture as in [3] to learn a regression function $\Gamma_{\text{self}}(\boldsymbol{q}) : \mathbb{R}^n \to \mathbb{R}$ to approximate the minimal distance between the robot's links, where

$$\nabla_{\boldsymbol{q}}\Gamma_{\text{self}}(\boldsymbol{q}) = \left[\frac{\partial\Gamma_{\text{self}}(\boldsymbol{q})}{\partial q_1}, \ldots, \frac{\partial\Gamma_{\text{self}}(\boldsymbol{q})}{\partial q_n}\right]^T$$

It represents a vector field defined in the joint space, providing the direction to potential self-collisions. The intuitive idea is to employ neural networks to approximate functions that are difficult to express explicitly and obtain their gradient field.

We perform a uniform sampling within the joint limits and employ MoveIt to collect a synthetic dataset. Each sample contains the joint angles and the minimal link distance. The dataset contains half a million entries, with 16% of configurations resulting in collisions.

### D. Incorporating Collision Avoidance Into a QP

To enforce collision avoidance, we can impose constraints to ensure that the safety distance $d_{r,r}$ between links and $d_{r,o}$ between links and the environment are satisfied. We linearize the distance function $\Gamma$ with respect to the $(\boldsymbol{q}, p)$ state as follows:

$$\begin{aligned}&\Gamma_{\text{env}}(\boldsymbol{q} + (\dot{\boldsymbol{q}}_d + \dot{\boldsymbol{q}}_c)dt, p + \dot{p}dt) \cong \Gamma_{\text{env}}(\boldsymbol{q}, p)\\ &\quad + \nabla_{\boldsymbol{q}}\Gamma_{\text{env}}(\boldsymbol{q}, p)^T(\dot{\boldsymbol{q}}_d + \dot{\boldsymbol{q}}_c)dt + \nabla_p\Gamma(\boldsymbol{q}, p)^T\dot{p}dt \geq d_{r,o} \quad (5)\\ &\Gamma_{\text{self}}(\boldsymbol{q} + (\dot{\boldsymbol{q}}_d + \dot{\boldsymbol{q}}_c)dt) \cong \Gamma_{\text{self}}(\boldsymbol{q})\\ &\quad + \nabla_{\boldsymbol{q}}\Gamma_{\text{self}}(\boldsymbol{q})^T(\dot{\boldsymbol{q}}_d + \dot{\boldsymbol{q}}_c)dt \geq d_{r,r} \quad (6)\end{aligned}$$

therefore, the obstacle avoidance optimization problem in joint space can be formulated as:

$$\min_{\dot{\boldsymbol{q}}_c} \|\dot{\boldsymbol{q}}_c\|_Q + \|J\dot{\boldsymbol{q}}_c\|_R \quad (7)$$

subject to:

$$\begin{aligned}&\nabla_{\boldsymbol{q}}\Gamma_{\text{env}}^T\dot{\boldsymbol{q}}_c \geq \frac{d_{r,o} - \Gamma_{\text{env}}}{dt} - \nabla_{\boldsymbol{q}}\Gamma_{\text{env}}^T\dot{\boldsymbol{q}}_d - \nabla_p\Gamma_{\text{env}}^T\dot{p}\\ &\nabla_{\boldsymbol{q}}\Gamma_{\text{self}}^T\dot{\boldsymbol{q}}_c \geq \frac{d_{r,r} - \Gamma_{\text{self}}}{dt} - \nabla_{\boldsymbol{q}}\Gamma_{\text{self}}^T\dot{\boldsymbol{q}}_d\\ &\dot{q}_{\min} - \dot{\boldsymbol{q}}_d \leq \dot{\boldsymbol{q}}_c \leq \dot{q}_{\max} - \dot{\boldsymbol{q}}_d\end{aligned}$$

where $\nabla_p\Gamma_{\text{env}} = [\nabla_p\Gamma_1, \nabla_p\Gamma_2, \ldots, \nabla_p\Gamma_m] \in \mathbb{R}^{3\times m}$ denotes the gradient of the predicted minimum distance vector with respect to the query point, $\dot{p}$ denotes the velocity of obstacles, $dt$ denotes the control period and $\dot{\boldsymbol{q}}_d$ represents the expected joint velocity. Generally, it can be defined as:

$$\dot{\boldsymbol{q}}_d = \begin{cases} J^T\left(JJ^T + \epsilon I\right)\log({}^wT_e{}^wT_c{}^{-1}), & (\text{Cartesian-space})\\ \eta\frac{q_f - q_0}{\|q_f - q_0\|}, & (\text{Joint-space})\end{cases} \quad (8)$$

where $I \in \mathbb{R}^{6\times6}$ is the identity matrix, $\epsilon$ is a small positive constant, and $\eta$ is a positive amplitude constant.

Due to the difficulty in accurately measuring the current control period $dt$, we adopt the upper bounds of $\frac{d_{r,o}-\Gamma_{\text{env}}}{dt}$ and $\frac{d_{r,r}-\Gamma_{\text{self}}}{dt}$, denoted as $\xi_{\text{env}}(d_{r,o} - \Gamma_{\text{env}})$ and $\xi_{\text{self}}(d_{r,r} - \Gamma_{\text{self}})$ respectively, to eliminate the dependency on $dt$ for more robust control. Here, $0 < \xi_{\text{env}}, \xi_{\text{self}} \leq \frac{1}{dt}$. Therefore, the constraints can be reformulated as follows:

$$\begin{aligned}&\nabla_{\boldsymbol{q}}\Gamma_{\text{env}}^T\dot{\boldsymbol{q}}_c \geq \xi_{\text{env}}\left(d_{r,o} - \Gamma_{\text{env}}\right) - \nabla_{\boldsymbol{q}}\Gamma_{\text{env}}^T\dot{\boldsymbol{q}}_d - \nabla_p\Gamma_{\text{env}}^T\dot{p}\\ &\nabla_{\boldsymbol{q}}\Gamma_{\text{self}}^T\dot{\boldsymbol{q}}_c \geq \xi_{\text{self}}\left(d_{r,r} - \Gamma_{\text{self}}\right) - \nabla_{\boldsymbol{q}}\Gamma_{\text{self}}^T\dot{\boldsymbol{q}}_d\\ &\dot{\boldsymbol{q}}_{\min} - \dot{\boldsymbol{q}}_d \leq \dot{\boldsymbol{q}}_c \leq \dot{\boldsymbol{q}}_{\max} - \dot{\boldsymbol{q}}_d\end{aligned}$$

We demonstrate that these constraints can prevent both environmental and self-collisions, ensuring that the minimum distance between the robot and the environment is not less than $d_{r,o}$, and that the minimum distance between the robot's links, is not less than $d_{r,r}$. The proof is provided in the Appendix.

As shown in the proof, the minimal distance functions satisfy $\Gamma(t) \geq d_s + e^{-\xi t}(\Gamma_0 - d_s)$, where $d_s$ adjusts the safety distance and $\xi$ controls the rate at which the robot approaches this safety distance. With a larger $\xi$, there exists more solution but the robot may approach the obstacles more quickly. Conversely, a smaller $\xi$ yields a more conservative solution, keeping the robot at a safer distance. We introduce a dynamic adjustment strategy for these hyper-parameters based on the dot product between $\dot{\boldsymbol{q}}_c$ and $\dot{\boldsymbol{q}}_d$, which represents the proximity to local minima. The parameters $\xi_{\text{env}}$ and $d_{r,o}$ are defined as:

$$\begin{aligned}\xi_{\text{env}} &= \sigma(\langle\dot{\boldsymbol{q}}_c, \dot{\boldsymbol{q}}_d\rangle, 1, 0.5, -0.2, 10)\xi_0\\ d_{r,o} &= \sigma(\langle\dot{\boldsymbol{q}}_c, \dot{\boldsymbol{q}}_d\rangle, 0.2, 1, -0.2, 100)d_0\end{aligned} \quad (9)$$

where $\xi_0$ and $d_0$ represent the initial values, and $\sigma$ denotes the parameterized Sigmoid function, defined as:

**Algorithm 1:** Collision-Free Motion Generation.

**Input:** Current joint states: $\boldsymbol{q}_{\text{cur}}$
Environment point cloud: $\mathcal{O}$
Optional obstacles velocity: $\dot{p}$
Link SDFs neural networks: $NN_{\text{links}}$
Self-collision neural networks: $NN_{\text{self}}$
**Output:** Collision-free joint motion: $\dot{\boldsymbol{q}}_d + \dot{\boldsymbol{q}}_c$

**while** *not arrived* **do**
  Update environment point cloud: $\mathcal{O}$
  Update expected joint motion: $\dot{\boldsymbol{q}}_d$
  Forward kinematics: $\text{FK}(\boldsymbol{q}_{\text{cur}})$
  Jacobian matrix: $J(\boldsymbol{q}_{\text{cur}})$
  **foreach** $NN$ *in* $NN_{links}$ **do**
    Nearest pair points in the $k$-th link: $(p, p_k)$
    Gradient of distance function: $\nabla_{\boldsymbol{q}}\Gamma_{\text{env}}$
  Gradient of self-collision function: $\nabla_{\boldsymbol{q}}\Gamma_{\text{self}}$
  Optimize $\dot{\boldsymbol{q}}_c$ by formulation (7)
  Update control duration: $dt$
  Servo: $\boldsymbol{q}_{\text{cur}} = \boldsymbol{q}_{\text{cur}} + (\dot{\boldsymbol{q}}_d + \dot{\boldsymbol{q}}_c)dt$

$\sigma(x, y_{\min}, y_{\max}, x_{\text{mid}}, k) = y_{\min} + \frac{y_{\max} - y_{\min}}{1+\exp(-k(x - x_{\text{mid}}))}$. The flowchart of collision-free motion generation is shown as the Algorithm 1.

## IV. Experiments Validation

In this section, we first perform a grid search to determine the appropriate network structure for fitting link SDFs. We then validate the performance of our algorithm through both simulation and physical experiments. For the simulated experiments, we use PyBullet to compute distances as the ground truth. Physical experiments are performed using the 6-DOF JAKA ZU7-1200 robot. Scene point clouds are captured with two hand-to-eye Intel RealSense RS485 cameras. The proposed algorithm is implemented in Python and executed on an AMD R4600H 2.80 GHz CPU with 16GB RAM and an RTX 1650 GPU.

### A. Network Structure of Link SDFs

The simple fully-connected MLP was used as the network architecture to learn link SDFs. We utilized grid search to explore the network depth ($D$) and the number of hidden layer nodes ($N$) to obtain the tradeoff between performance and efficiency.

The results are presented in Table I, with the chosen link illustrated in Fig. 4 for testing. These results are structured according to distance regression loss (Mean Absolute Error, measured in centimeters), normal regression loss, and computation time. The computation time in the table represents the result of forward inference for 10,000 points. The result indicates that link SDFs exhibit low demands on network complexity. Even a simple structure can provide sufficient fitting while improving computational efficiency. For real-time optimization, where forward inference for all $m$ links is necessary, only a shallow network with few nodes can be adopted. As network complexity increases, training becomes more difficult, with a higher risk of overfitting.

TABLE I
A Comparison of Networks With Different $D$ and $N$

| D/N | 32 | 64 | 128 | 256 |
|---|---|---|---|---|
| 3 | 0.54, 0.092 (0.231 ms) | 0.21, 0.075 (0.378 ms) | 0.17, 0.072 (0.635 ms) | 0.13, 0.067 (1.760 ms) |
| 5 | 0.23, 0.087 (0.360 ms) | 0.12, 0.087 (0.604 ms) | 0.08, 0.080 (1.328 ms) | 0.06, 0.064 (4.076 ms) |
| 7 | 0.29, 0.460 (0.477 ms) | 0.10, 0.113 (0.904 ms) | 0.08, 0.088 (2.015 ms) | 0.08, 0.080 (6.374 ms) |
| 9 | 0.47, 0.564 (0.592 ms) | 0.18, 0.680 (1.117 ms) | 0.05, 0.291 (2.709 ms) | 0.03, 0.091 (8.677 ms) |

TABLE II
Performance of the Learned Link SDFs

| Link ID | MAE(mm) | SD(mm) | Max Absolute Error(mm) |
|---|---|---|---|
| 1 | **3.45** / 3.86 | **3.15** / 5.07 | **22.90** / 40.59 |
| 2 | **2.68** / 4.00 | **3.49** / 5.19 | **19.19** / 43.01 |
| 3 | **2.51** / 4.46 | **3.02** / 5.79 | **16.76** / 40.54 |
| 4 | **2.16** / 5.89 | **2.81** / 7.76 | **17.17** / 66.40 |
| 5 | **2.59** / 7.19 | **3.31** / 9.48 | **24.61** / 79.66 |
| 6 | **3.72** / 10.08 | **4.71** / 13.48 | **56.58** / 188.26 |
| 7 | **3.83** / 12.62 | **4.92** / 16.64 | **60.22** / 148.39 |
| 8 | **3.37** / 14.86 | **4.46** / 19.45 | **36.46** / 143.03 |

Therefore, we chose a network with 5 fully connected layers, with all hidden layers being 64-dimensional.

### B. Quality of Modeling Collision

We compare the link SDFs with Neural-JSDF [3] to validate the accuracy of representing the robot's geometry. Mean Absolute Error (MAE), Standard Deviation (SD), and maximum absolute error are utilized to assess the performance of distance regression. The test points are sourced from the training dataset of Neural-JSDF, which favors Neural-JSDF.

The comparison results are presented in Table II, with units in millimeters (mm). The outcomes of the link SDFs are highlighted in bold, while the corresponding results represent those of Neural-JSDF. The link SDFs provide improved fitting accuracy for each link, addressing the challenge observed in Neural-JSDF, where fitting accuracy declines for distant links. Additionally, we evaluated the efficiency of the link SDFs and Neural-JSDF in querying both the distance and the gradient in scenarios involving 10,000 points. The computation times were 9.05 ms and 17.42 ms, respectively.

We further validated the effectiveness of the self-collision network in terms of the accuracy of predicting collisions, represented as $\text{sign}(\Gamma_{\text{self}}(\boldsymbol{q}))$ for 10,000 randomly sampled configurations. The self-collision network can achieve an accuracy of 98.7%.

### C. Simulation Experiments

We demonstrate the capability of our algorithm by applying it to several different scenarios, where the default motion results in environmental and self-collisions. The initial joint state (Cartesian pose) and the target joint state (Cartesian pose) are

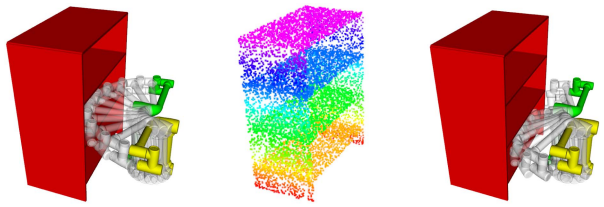


Fig. 5. Static obstacle avoidance scenario. (a) The default joint motion results in a collision with the shelf. (b) The scene point cloud used for the collision avoidance algorithm. (c) The planned collision-free joint motion.

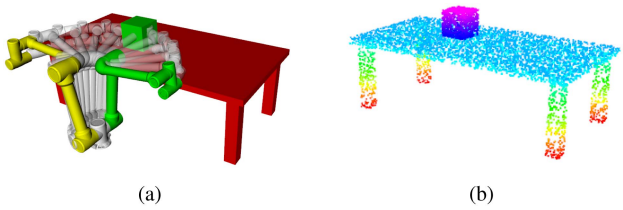


Fig. 6. Dynamic obstacle avoidance scenario. (a) The default joint motion results in a collision with the obstacles. (b) The scene point cloud used for the collision avoidance algorithm.

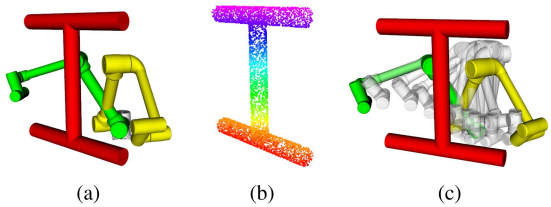


Fig. 7. Planning scenario in Cartesian space. (a) The initial and the target poses. (b) the point cloud used for the collision avoidance algorithm. (c) The planned collision-free joint motion.

represented in green and yellow, respectively. The scenes are depicted using point clouds. The tested scenarios are as follows:

1) *S1:* static obstacles. The robot moves from its initial configuration to a target configuration, with a bookshelf positioned 0.5 meters away from the robot base, as shown in Fig. 5.
2) *S2:* static and dynamic obstacles. The robot moves from its initial configuration to a target configuration while a cube, with a length of 0.2 meters, performs reciprocating motion at a velocity of 0.05 m/s, as shown in Fig. 6.
3) *S3:* target Cartesian pose reaching. The robot moves to a target Cartesian pose while avoiding a static obstacle, as shown in Fig. 7.

We conducted 10 repeated experiments for *S1* and *S2*, resulting in average control periods of 9.40 ms and 9.45 ms, respectively. These results indicate that the proposed algorithm operates at approximately 100 Hz. We further evaluated the impact of incorporating obstacle velocity on collision-free planning in Scenario *S2*. The minimum distances between the robot and obstacles are shown in Fig. 8(a), demonstrating that including obstacle velocity in the environmental collision avoidance constraint is essential for ensuring collision-free motion.

For Scenario *S3*, we compared algorithms with and without self-collision avoidance constraints, marked as "/w" and "/wo"

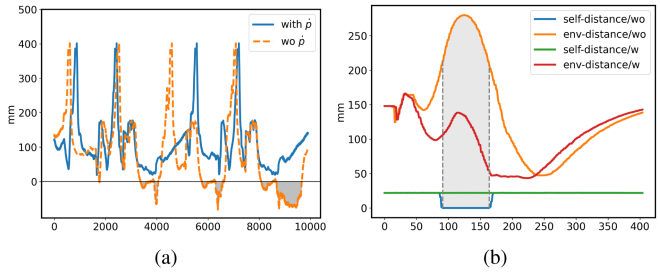


Fig. 8. (a) The curves of the env-distance, with the gray areas indicating instances of environmental collisions in the joint motion generated by the algorithm when obstacle velocity is not considered. (b) The curves of env-distance and self-distance, with the gray areas indicating instances of self-collisions in the joint motion generated by the algorithm without self-collision constraints.

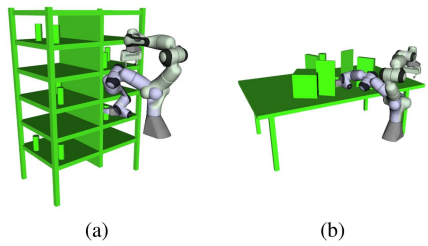


Fig. 9. Selected scenes used to evaluate the performance from MotionBenchMaker. The initial joint state is shown in light-green, and the goal joint state is in light-purple. (a) Thin Bookshelf (b) Table Pick.

respectively, as shown in Fig. 8(b). This figure illustrates the minimum distance between the robot and the environment (referred to as env-distance) and between the robot's links (referred to as self-distance). The algorithm with self-collision constraints operates at a frequency of approximately 50 Hz. Given the robot's structure, the maximum self-collision distance is 22 mm, with a self-collision distance of zero indicating actual collisions. From Fig. 8(b), it is evident that the algorithm without self-collision constraints maintains a greater distance from obstacles but may lead to self-collisions. In contrast, the algorithm with self-collision constraints successfully avoids collisions both with the environment and within the robot itself.

### D. Simulation Benchmark

To comprehensively evaluate the performance of our algorithm, we selected 200 predefined problems across two different scenes from the MotionBenchMaker [25] for the Panda robot. In each problem, the initial and target joint configurations are specified. The selected test scenes are Thin Bookshelf and Table Pick, as shown in Fig. 9. These problems are challenging because gradient-based optimization is prone to get trapped in local minima. We compare the performance of our algorithm with CHOMP [6] and the QPIK method presented in Neural-JSDF [3]. It is important to note that these problems are specifically designed to benchmark joint-space planning performance, where initial and target joint configurations are defined for each task. However, QPIK can only perform Cartesian-space planning. Therefore, we compare our algorithm with CHOMP for joint-space planning and with QPIK for Cartesian-space planning, where we convert the target configurations into target

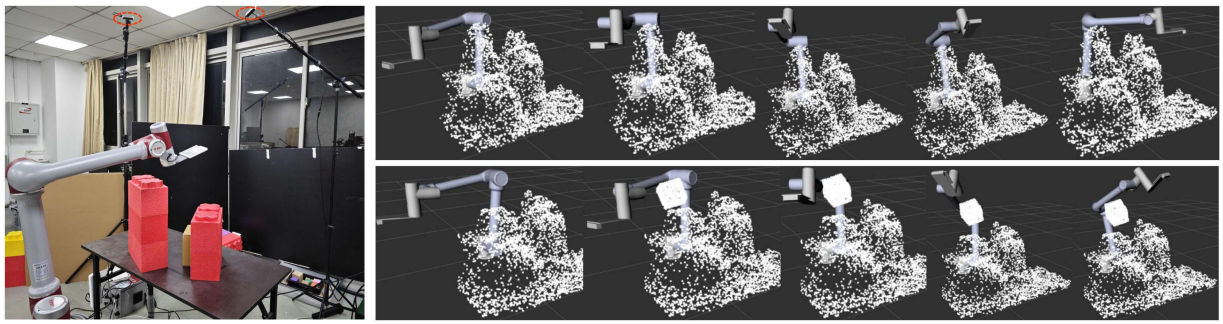

Fig. 10. Physical Experiments: (Left) Experimental setup where two hand-to-eye cameras (highlighted with red circles) are used to acquire the scene's point cloud. The scenes are arbitrarily constructed using building blocks. (Top right) Joint motion of the robot in a static environment. (Bottom right) Joint motion of the robot in a static environment with a dynamic obstacle.

TABLE III
PERFORMANCE COMPARISON OF BENCHMARK

| Scene ID | Success (self-collision) rate (%) | | | |
|---|---|---|---|---|
| | Joint-space | | Cartesian-space | |
| | CHOMP | Ours | QPIK | Ours |
| Bookshelf | - | **60(0)** | 90(74.4) | **98(0)** |
| Table Pick | 11(0) | **59(0)** | 56(51.8) | **74(0)** |

poses. These scenes present greater challenges in joint space compared to Cartesian space.

For each scenario, the algorithm terminates either upon successfully reaching the target or exceeding the maximum iteration limit of 5000. In joint-space planning, success is defined as achieving a 2-norm distance between the current and target joint configurations of less than 0.05 radians. In Cartesian-space planning, success is defined as the sum of the absolute values of the twist between the current and target poses being less than 0.1. The initial parameters of our method were set to $\xi_0 = 1.0$ and $d_0 = 0.1$ m, while CHOMP used the default parameters in MoveIt. The self-collision rate is calculated as the proportion of self-collisions among all successful cases.

The comparison results are presented in Table III, where "$-$" indicates that CHOMP was unable to solve the Thin Bookshelf scenarios. These results demonstrate that our algorithm outperforms others in both joint-space and Cartesian-space planning. Due to the absence of self-collision constraints, there exist self-collisions during the motion generated by QPIK.

### *E. Real Robot Experiments*

We further validated the approach on a real JAKA Zu7-1200. The experimental setup is shown on the left side of Fig. 10. The environmental information is represented by the scene point cloud, which is captured by two RealSense hand-to-eye depth cameras. However, as the robot moves, the cameras' field of view may become obstructed, causing missing points in the point cloud and introducing noisy points generated by the robot itself. Therefore, before each robot movement, we first use the cameras to capture a static point cloud to represent the static scene information. This static scene can be updated as needed before each movement. Additionally, the pose of the dynamic obstacle was detected using AprilTag markers, which were then used to update the point cloud of the dynamic obstacle, integrating it into the overall scene point cloud. The top-right and bottom-right sections of Fig. 10 respectively illustrate collision-free motions in scenarios without dynamic obstacles and with a dynamic cube obstacle. More details can be found in the supplementary media.

## V. CONCLUSION

This letter presents a planner designed to generate collision-free motion based on perceptual point clouds, capable of solving scenarios with up to 10,000 points within 10 milliseconds. We reformulate reactive motion planning as a QP problem, incorporating constraints for both environmental and self-collisions. Implicit signed distance functions are modeled using neural networks, and we leverage forward kinematics to eliminate the non-linear mappings between configuration and Cartesian space, enhancing fitting accuracy with the derived analytical gradient of the distance. Additionally, we introduce dynamic adjustment of hyperparameters based on proximity to local minima, allowing effective handling of more densely cluttered environments. The effectiveness of our approach is validated through both simulations and physical experiments. While we have demonstrated that the linearized constraints can maintain the safety distance, sensor measurements are inherently noisy and learning-based signed distance functions introduce prediction uncertainties. Future work will focus on developing methods to address these uncertainties to provide stronger safety guarantees. Furthermore, investigating techniques to handle local minima in more complex concave environments will be a valuable direction for further research.

## APPENDIX

We prove that these linearized constraints ensure that no environmental or self-collisions occur during the manipulator's movement. The minimum distance $d_k$ between the $k$th link and the obstacles is illustrated in Fig. 11. For clarity, we define $n_{o,k}$ as the unit vector pointing from $p_k$ to $p_o$.

Based on the gradient of the distance function $d_k$ with respect to $\boldsymbol{q}$ and $p$, we can deduce that the term $\nabla_{\boldsymbol{q}}\Gamma_{\text{env}}^T(\dot{\boldsymbol{q}}_c + \dot{\boldsymbol{q}}_d)$ represents the projection of the velocity $\dot{p}_k$ onto $n_{k,o}$, and $\nabla_p\Gamma_{\text{env}}^T\dot{p}$ denotes the projection of the velocity $\dot{p}_o$ onto $n_{o,k}$. Therefore, $\dot{\Gamma}_{\text{env}} = \nabla_{\boldsymbol{q}}\Gamma_{\text{env}}^T(\dot{\boldsymbol{q}}_c + \dot{\boldsymbol{q}}_d) + \nabla_p\Gamma_{\text{env}}^T\dot{p}$ represents the

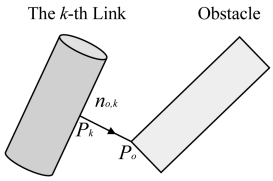


Fig. 11. Minimum distance between the $k$th link and an obstacle, where the witness point pairs are denoted as $p_k, p_o$.

separation velocity between the link and the obstacle. The linearized collision avoidance constraints take the general form $\dot{\Gamma}(t) \geq \xi(d_s - \Gamma(t))$, where $d_s$ denotes the safety threshold, and $\Gamma$ is a function of time $t$. The general solution to the non-homogeneous differential equation $\dot{\Gamma}(t) = \xi(d_s - \Gamma(t))$ is given by:

$$\Gamma(t) = d_s + e^{-\xi t}\left(\Gamma_0 - d_s\right) \tag{10}$$

where $\Gamma_0$ is the initial minimal distance at $t = 0$. From this, the solution can be expressed as:

$$\Gamma(t) \geq d_s + e^{-\xi t}\left(\Gamma_0 - d_s\right) \tag{11}$$

From the solution, if the initial distance $\Gamma_0 \geq d_s$, we can conclude that $\Gamma(t) \geq d_s$ for all $t \geq 0$. Thus, under the linearized collision avoidance constraints, the minimum distance will always remain greater than or equal to the safe threshold $d_s$.

## REFERENCES


[1] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Aggressive driving with model predictive path integral control," in *Proc. 2016 IEEE Int. Conf. Robot. Automat.*, 2016, pp. 1433–1440.

[2] M. Bhardwaj et al., "STORM: An integrated framework for fast joint-space model-predictive control for reactive manipulation," in *Proc. Conf. Robot Learn.*, 2022, pp. 750–759.

[3] M. Koptev, N. Figueroa, and A. Billard, "Neural joint space implicit signed distance functions for reactive robot manipulator control," *IEEE Robot. Automat. Lett.*, vol. 8, no. 2, pp. 480–487, Feb. 2023.

[4] B. Liu, G. Jiang, F. Zhao, and X. Mei, "Collision-free motion generation based on stochastic optimization and composite signed distance field networks of articulated robot," *IEEE Robot. Automat. Lett.*, vol. 8, no. 11, pp. 7082–7089, Nov. 2023.

[5] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "STOMP: Stochastic trajectory optimization for motion planning," in *Proc. 2011 IEEE Int. Conf. Robot. Autom.*, 2011, pp. 4569–4574.

[6] M. Zucker et al., "CHOMP: Covariant hamiltonian optimization for motion planning," *Int. J. Robot. Res.*, vol. 32, no. 9/10, pp. 1164–1193, 2013.

[7] J. Schulman et al., "Motion planning with sequential convex optimization and convex collision checking," *Int. J. Robot. Res.*, vol. 33, no. 9, pp. 1251–1270, 2014.

[8] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *Int. J. Robot. Res.*, vol. 5, no. 1, pp. 90–98, 1986.

[9] J. Haviland and P. Corke, "NEO: A novel expeditious optimisation algorithm for reactive motion control of manipulators," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 1043–1050, Apr. 2021.

[10] G. Williams, A. Aldrich, and E. A. Theodorou, "Model predictive path integral control: From theory to parallel computation," *J. Guid. Control Dyn.*, vol. 40, no. 2, pp. 344–357, 2017.

[11] C. Pezzato, C. Salmi, M. Spahn, E. Trevisan, J. Alonso-Mora, and C. H. Corbato, "Sampling-based model predictive control leveraging parallelizable physics simulations," Jul. 2023, *arXiv:2307.09105*.

[12] M. Koptev, N. Figueroa, and A. Billard, "Reactive collision-free motion generation in joint space via dynamical systems and sampling-based MPC," *Int. J. Robot. Res.*, vol. 43, pp. 2049–2069, 2024.

[13] B. Sundaralingam et al., "CuRobo: Parallelized collision-free minimum-jerk robot motion generation," in *Proc. 2023 IEEE Int. Conf. Robot. Automat. (ICRA)*, 2023, pp. 8112–8119.

[14] M. Macklin, K. Erleben, M. Müller, N. Chentanez, S. Jeschke, and Z. Corse, "Local optimization for robust signed distance field collision," *Proc. ACM Comput. Graph. Interactive Techn.*, vol. 3, no. 1, pp. 1–17, 2020.

[15] J. C. Carr et al., "Reconstruction and representation of 3D objects with radial basis functions," in *Proc. 28th Annu. Conf. Comput. Graph. Interactive Techn.*, 2001, pp. 67–76.

[16] C. Ramsey, Z. Kingston, W. Thomason, and L. E. Kavraki, "Collision-affording point trees: SIMD-Amenable nearest neighbors for fast motion planning with pointclouds," in *Proc. Robot.: Sci. Syst.*, Delft, Netherlands, Jul. 2024.

[17] P. Liu, K. Zhang, D. Tateo, S. Jauhri, J. Peters, and G. Chalvatzaki, "Regularized deep signed distance fields for reactive motion generation," in *Proc. 2022 IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 6673–6680.

[18] V. Vasilopoulos, S. Garg, P. Piacenza, J. Huh, and V. Isler, "RAMP: Hierarchical reactive motion planning for manipulation tasks using implicit signed distance functions," in *Proc. 2023 IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2023, pp. 10551–10558.

[19] C. Quintero-Pena, W. Thomason, Z. Kingston, A. Kyrillidis, and L. E. Kavraki, "Stochastic implicit neural signed distance functions for safe motion planning under sensing uncertainty," in *Proc. 2024 IEEE Int. Conf. Robot. Automat.*, 2024, pp. 2360–2367.

[20] N. B. Figueroa Fernandez, S. S. Mirrazavi Salehian, and A. Billard, "Multi-arm self-collision avoidance: A sparse solution for a Big Data problem," in *Proc. 3rd Mach. Learn. Plan. Control Robot Motion Workshop*, 2018.

[21] S. S. Mirrazavi Salehian, N. Figueroa, and A. Billard, "A unified framework for coordinated multi-arm motion planning," *Int. J. Robot. Res.*, vol. 37, no. 10, pp. 1205–1232, 2018.

[22] M. Koptev, N. Figueroa, and A. Billard, "Real-time self-collision avoidance in joint space for humanoid robots," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 1240–1247, Apr. 2021.

[23] T. Noël, T. Flayols, J. Mirabel, J. Carpentier, and N. Mansard, "A hybrid collision model for safety collision control," in *Proc. 2021 IEEE Int. Conf. Robot. Automat.*,, 2021, pp. 1722–1728.

[24] C. W. Wampler, "Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods," *IEEE Trans. Syst., Man, Cybern.*, vol. 16, no. 1, pp. 93–101, Jan. 1986.

[25] C. Chamzas et al., "MotionBenchMaker: A tool to generate and benchmark motion planning datasets," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 882–889, Apr. 2022.